

BAB 3. JSON WEB TOKEN (JWT)

3.1 CAPAIAN PEMBELAJARAN

Kode CPL	: CPL08
Nama CPL	: Menjelaskan definisi dan ruang lingkup API
Kode CPMK	: 1
Nama CPMK	: Mahasiswa mampu menjelaskan definisi dan ruang lingkup API
Nama Sub-CPMK	: Menjelaskan definisi dan ruang lingkup API

3.2 TUJUAN PERCOBAAN

Mahasiswa mampu menjelaskan definisi dan ruang lingkup API.

3.3 TINJAUAN PUSTAKA

JWT adalah singkatan dari "JSON Web Token". Ini adalah cara yang ringkas dan aman untuk URL untuk mewakili klaim yang akan ditransfer antara dua pihak. Klaim ini biasanya digunakan untuk menyandikan informasi tentang pengguna yang diautentikasi dan metadata tambahan. JWT biasanya digunakan untuk tujuan otentikasi dan otorisasi dalam pengembangan web. JWT terdiri dari tiga bagian: header, payload, dan tanda tangan. Header biasanya terdiri dari jenis token (JWT) dan algoritma penandatanganan yang digunakan, sedangkan payload berisi klaim.

Tanda tangan dibuat dengan mengambil header yang disandikan, payload yang disandikan, rahasia, dan algoritma yang ditentukan dalam header, dan menandatangani. JWT sering digunakan dalam protokol autentikasi seperti OAuth 2.0 dan OpenID Connect, yang dapat digunakan untuk mengirimkan informasi antar pihak dengan aman. Mereka juga digunakan dalam mekanisme autentikasi tanpa kewarganegaraan karena server dapat memverifikasi integritas token tanpa harus menyimpan status sesi apa pun. Secara keseluruhan, JWT menyediakan cara untuk mengirimkan informasi antar pihak dengan aman dan banyak digunakan dalam pengembangan aplikasi web modern.

3.4 ALAT DAN BAHAN

- Kebutuhan peralatan
 1. Komputer PC/Notebook
 2. Visual Studio 2022

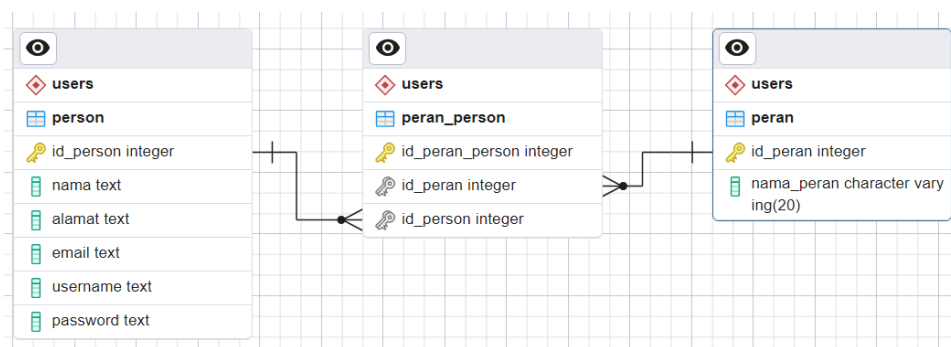
3. Postman
4. Postgresql dan PgAdmin-4
- Kebutuhan bahan praktikum
 1. -

3.5 PROSEDUR KERJA

Percobaan 1: Menambahkan JWT Authentikasi

Langkah-langkah percobaan:

1. Buatlah dua tabel baru dengan nama “peran” dan “peran_person” dengan ERD sebagai berikut.



2. Isilah table “peran” dan “peran_user” dengan beberapa baris data dengan memperhatikan foreign key yang berlaku.
3. Ubahlah file “appSetting.json” menjadi sebagai berikut untuk menangani JWT.

```

ma: https://json.schemastore.org/appsettings.json
1  {
2    "ConnectionStrings": {
3      "WebApiDatabase": "Host=127.0.0.1; Port=5432; Database=pbo2023; Username=istiyadi; Password=123456"
4    },
5    "Logging": {
6      "LogLevel": {
7        "Default": "Information",
8        "Microsoft.AspNetCore": "Warning"
9      }
10   },
11   "Jwt": {
12     "Key": "zzaasdv234234*6%6*6*hjsdfb2%%zzaaqwscvdefrgbnh",
13     "Issuer": "https://localhost:7162",
14     "Audience": "https://localhost:7162"
15   },
16   "AllowedHosts": "*"
17 }
  
```

4. Buatlah file “Login.cs” dan “LoginContext.cs” pada direktori “Models” dengan isi file sebagai berikut:

```
1 namespace PercobaanApil.Models
2 {
3     4 references
4     public class Login
5     {
6         1 reference
7         public int id_person { get; set; }
8         1 reference
9         public string nama { get; set; }
10        1 reference
11        public string alamat { get; set; }
12        1 reference
13        public string email { get; set; }
14        0 references
15        public int id_peran { get; set; }
16        0 references
17        public string nama_peran { get; set; }
18    }
19 }
```

```

PercobaanApi1
1 using Microsoft.IdentityModel.Tokens;
2 using Npgsql;
3 using PercobaanApi1.Helpers;
4 using System.IdentityModel.Tokens.Jwt;
5 using System.Security.Claims;
6 using System.Text;
7
8 namespace PercobaanApi1.Models
9 {
10     3 references
11     public class LoginContext
12     {
13         private string __constr;
14         private string __ErrorMsg;
15         1 reference
16         public LoginContext(string pConstr)
17         {
18             __constr = pConstr;
19         }
20         1 reference
21         public List<Login> Authentifikasi(string p_username, string p_password, IConfiguration p_config)
22         {
23             List<Login> list1 = new List<Login>();
24             string query = string.Format(@"SELECT ps.id_person, ps.nama, ps.alamat, ps.email,
25             pp.id_peran, p.nama_peran
26             FROM users.person ps
27             INNER JOIN users.peran_person pp ON ps.id_person=pp.id_person
28             INNER JOIN users.peran p ON pp.id_peran=p.id_peran
29             where ps.username='{0}' and ps.password='{1}'; ", p_username, p_password);
30             SqlDBHelper db = new SqlDBHelper(this.__constr);
31             try
32             {
33                 NpgsqlCommand cmd = db.getNpgsqlCommand(query);
34                 NpgsqlDataReader reader = cmd.ExecuteReader();
35                 while (reader.Read())
36                 {
37                     list1.Add(new Login()
38                     {
39                         id_person = int.Parse(reader["id_person"].ToString()),
40                         nama = reader["nama"].ToString(),
41                         alamat = reader["alamat"].ToString(),
42                         email = reader["email"].ToString(),
43                         id_peran = int.Parse(reader["id_peran"].ToString()),
44                         nama_peran = reader["nama_peran"].ToString(),
45                         token = GenerateJwtToken(p_username, p_password, p_config)
46                     });
47                 }
48                 cmd.Dispose();
49                 db.closeConnection();
50             }
51             catch (Exception ex)
52             {
53                 __ErrorMsg = ex.Message;
54             }
55             return list1;
56         }
57     }
58     1 reference
59     private string GenerateJwtToken(string namaUser, string peran, IConfiguration pConfig)
60     {
61         var securityKey = new SymmetricSecurityKey(Encoding.ASCII.GetBytes(pConfig["Jwt:Key"]));
62         var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256);
63         var claims = new[]
64         {
65             new Claim(ClaimTypes.NameIdentifier, namaUser),
66             new Claim(ClaimTypes.Role, peran)
67         };
68         var token = new JwtSecurityToken(pConfig["Jwt:Issuer"],
69         pConfig["Jwt:Audience"],
70         claims,
71         expires: DateTime.Now.AddMinutes(15),
72         signingCredentials: credentials);
73         return new JwtSecurityTokenHandler().WriteToken(token);
74     }
75 }
76

```

5. Install library berikut ini dari Nuget
 - Microsoft.AspNetCore.Authentication.JwtBearer
6. Tambahkan Controller "LoginController.cs" sebagai berikut:

```

PercobaanApi1
1 using Microsoft.AspNetCore.Mvc;
2 using PercobaanApi.Models;
3
4 namespace PercobaanApi.Controllers
5 {
6     1 reference
7     public class LoginController : Controller
8     {
9         private string __constr;
10        private IConfiguration __config;
11        0 references
12        public LoginController(IConfiguration configuration)
13        {
14            __config = configuration;
15            __constr = configuration.GetConnectionString("WebApiDatabase");
16        }
17        0 references
18        public IActionResult Index()
19        {
20            return View();
21        }
22        [HttpPost("api/Login")]
23        0 references
24        public IEnumerable<Login> LoginUser(string namaUser, string password)
25        {
26            LoginContext context = new LoginContext(this.__constr);
27            List<Login> listLogin = context.Authentifikasi(namaUser, password, __config);
28            return (listLogin).ToArray();
29        }
30    }
31 }

```

7. Ubah file “PersonController.cs” menjadi sebagai berikut:

```

PercobaanApi1
1 using Microsoft.AspNetCore.Authorization;
2 using Microsoft.AspNetCore.Mvc;
3 using PercobaanApi.Models;
4
5 namespace PercobaanApi.Controllers
6 {
7     1 reference
8     public class PersonController : Controller
9     {
10        private string __constr;
11        0 references
12        public PersonController(IConfiguration configuration)
13        {
14            __constr = configuration.GetConnectionString("WebApiDatabase");
15        }
16        0 references
17        public IActionResult Index()
18        {
19            return View();
20        }
21        [HttpGet("api/person")]
22        0 references
23        public ActionResult<Person> ListPerson()
24        {
25            PersonContext context = new PersonContext(this.__constr);
26            List<Person> listPerson = context.ListPerson();
27            return Ok(listPerson);
28        }
29        [HttpPost("api/person_auth"), Authorize]
30        0 references
31        public ActionResult<Person> ListPersonWithAuth()
32        {
33            PersonContext context = new PersonContext(this.__constr);
34            List<Person> listPerson = context.ListPerson();
35            return Ok(listPerson);
36        }
37    }
38 }

```

8. Ubah source code “Program.cs” menjadi sebagai berikut:

```

PercobaanApi1
1 using Microsoft.AspNetCore.Authentication.JwtBearer;
2 using Microsoft.IdentityModel.Tokens;
3 using Microsoft.OpenApi.Models;
4 using System.Text;
5
6 var builder = WebApplication.CreateBuilder(args);
7
8 // Add services to the container.
9
10 builder.Services.AddControllers();
11 // Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
12 builder.Services.AddEndpointsApiExplorer();
13 builder.Services.AddSwaggerGen();
14
15 builder.Services.AddCors(p => p.AddPolicy("corsapp", builder =>
16 {
17     builder.WithOrigins("*").AllowAnyMethod().AllowAnyHeader();
18 }));
19
20 builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(options =>
21 {
22     options.TokenValidationParameters = new TokenValidationParameters
23     {
24         ValidateIssuer = true,
25         ValidateAudience = true,
26         ValidateLifetime = true,
27         ValidateIssuerSigningKey = true,
28         ValidIssuer = builder.Configuration["Jwt:Issuer"],
29         ValidAudience = builder.Configuration["Jwt:Audience"],
30         IssuerSigningKey = new SymmetricSecurityKey(Encoding.ASCII.GetBytes(builder.Configuration["Jwt:Key"]))
31     };
32 });
33
34
35 builder.Services.AddSwaggerGen(option =>
36 {
37     option.SwaggerDoc("v1", new OpenApiInfo { Title = "Demo API", Version = "v1" });
38     option.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
39     {
40         In = ParameterLocation.Header,
41         Description = "Please enter a valid token",
42         Name = "Authorization",
43         Type = SecuritySchemeType.Http,
44         BearerFormat = "JWT",
45         Scheme = "Bearer"
46     });
47     option.AddSecurityRequirement(new OpenApiSecurityRequirement
48     {
49         {
50             new OpenApiSecurityScheme
51             {
52                 Reference = new OpenApiReference
53                 {
54                     Type=ReferenceType.SecurityScheme,
55                     Id="Bearer"
56                 },
57             },
58             new string[]{}
59         },
60     });
61 });
62
63 var app = builder.Build();
64
65 // Configure the HTTP request pipeline.
66 if (app.Environment.IsDevelopment())
67 {
68     app.UseSwagger();
69     app.UseSwaggerUI();
70 }
71
72 app.UseHttpsRedirection();
73
74 app.UseAuthentication();
75
76 app.UseAuthorization();
77
78 app.MapControllers();
79
80 app.Run();
81
82

```

1. Jalankan aplikasi dan tampilkan hasilnya. Gunakan token (JWT token) authentication untuk mengakses method yang memerlukan. Gunakan Swagger dan juga Postman untuk melakukan autentifikasi dan mengakses data.

3.6 HASIL DAN ANALISIS DATA

Percobaan 1: Menambahkan JWT Autentikasi

- Hasil percobaan 1:

- Analisis:

3.7 KESIMPULAN

3.8 TUGAS MANDIRI

Gunakan kembali program yang telah kalian buat pada modul 2 dan tambahkan fitur login (1 user saja untuk admin) dan implementasikan JWT sebagai metode auth.

3.9 DAFTAR PUSTAKA

1. Brenda Jin, Saurabh Sahni, and Amir Shevat, “Designing Web APIs, Building APIs That Developers Love”, Published by O’Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472, 2018.
2. Marinki Spasojevic, Vladimir Pecanac, “Ultimate Asp.Net Core Web Api”, -
3. <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api?view=aspnetcore-7.0&tabs=visual-studio>